

**SIMATS ENGINEERING
ASSESSMENT TOOL 1**

COURSE CODE: CSA1401

COURSE NAME: COMPILER DESIGN

COURSE OUTCOME COVERED

CO3: *Analyze syntax-directed translation method using synthesized and inherited attributes to generate a Parser Tree. (BL4)*

Assessment Tool Weightage: 40%

Duration :60 Mins On Class

QUESTIONS:5

Total Marks:50

Annexure A

EXPERIMENTS

Code of Conduct:

I **Dinesh V (192524309)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **Dinesh V**

Question 1: Calculator Compiler Using YACC (10 Marks)

Problem Description: A calculator compiler evaluates arithmetic expressions during parsing using YACC. Design grammar for + and *, embed semantic actions, evaluate 3+4*5, and display the result.

(i) & (ii) YACC Specification and Lexer Implementation

1. YACC Grammar File (calc.y):

```
%{
#include <stdio.h>
#include <stdlib.h>

int yylex(void);
void yyerror(const char *s);
}%

%token NUMBER
%left '+'
%left '*'

%%

input:
    expr '\n' {
        printf("=====\n");
        printf("Computed Result: %d\n", $1);
        printf("=====\n");
        exit(0);
    }
;

expr:
    expr '+' expr { $$ = $1 + $3; }
  | expr '*' expr { $$ = $1 * $3; }
  | NUMBER      { $$ = $1; }
;

%%

void yyerror(const char *s) {
    fprintf(stderr, "Parsing Error: %s\n", s);
}

int main(void) {
    printf("Enter arithmetic expression (e.g., 3+4*5): ");
    yyparse();
    return 0;
}
```

2. Lexer File (calc.l):

```
%{
#include "y.tab.h"
#include <stdlib.h>
%}

%%
[0-9]+ { yylval = atoi(yytext); return NUMBER; }
[+|^*\n] { return yytext[0]; }
[ \t] ; /* Ignore whitespace */
. { printf("Unexpected character: %s\n", yytext); }
%%

int yywrap(void) { return 1; }
```

(iii) Step-by-Step Evaluation of '3+4*5'

The parser uses operator precedence declarations (%left '+' followed by %left '*') to resolve shift/reduce conflicts. Multiplication (*) has higher precedence than addition (+), ensuring that 4*5 is evaluated prior to adding 3.

Step	Input Token / Sub-expression	Action / Production Applied	Semantic Value (\$\$)
1	3	NUMBER -> expr	\$\$ = 3
2	4	NUMBER -> expr	\$\$ = 4
3	5	NUMBER -> expr	\$\$ = 5
4	4 * 5	expr '*' expr -> expr	\$\$ = 4 * 5 = 20
5	3 + 20	expr '+' expr -> expr	\$\$ = 3 + 20 = 23

(iv) Displayed Result after Parsing

```
C:\Windows\System32\cmd.e  ×  +  v
Microsoft Windows [Version 10.0.26200.8875]
(c) Microsoft Corporation. All rights reserved.

C:\Users\DINESH VS\Desktop\YACC\expl>flex expl.l
C:\Users\DINESH VS\Desktop\YACC\expl>bison -dy expl.y
C:\Users\DINESH VS\Desktop\YACC\expl>gcc lex.yy.c y.tab.c
C:\Users\DINESH VS\Desktop\YACC\expl>a
Enter an arithmetic expression: 3+4*5
Result = 23
```

Question 2: Construction and Traversal of Abstract Syntax Tree (AST) (10 Marks)

Problem Description: Define a YACC grammar to build a parse tree/AST for arithmetic expressions, illustrate the tree for 'a+b*c', and perform tree traversals.

(i) & (ii) YACC Grammar with AST Construction Actions (ast.y)

```
%{
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct Node {
    char label[10];
    struct Node *left;
    struct Node *right;
} Node;

Node* createNode(char* label, Node* left, Node* right) {
    Node* n = (Node*)malloc(sizeof(Node));
    strcpy(n->label, label);
    n->left = left;
    n->right = right;
    return n;
}

void printPostorder(Node* root) {
    if (!root) return;
    printPostorder(root->left);
    printPostorder(root->right);
    printf("%s ", root->label);
}

void printInorder(Node* root) {
    if (!root) return;
    if (root->left) printf("(");
    printInorder(root->left);
    printf("%s", root->label);
    printInorder(root->right);
    if (root->right) printf(")");
}

int yylex(void);
void yyerror(const char *s);
%}

%union {
    struct Node* node;
    char id[10];
}

%token <id> IDENTIFIER
%type <node> expr
%left '+'
```

```

%left '*'

%%

stmt: expr '\n' {
    printf("--- Abstract Syntax Tree Traversal Results ---\n");
    printf("Postorder Traversal (Bottom-Up Evaluation Order): ");
    printPostorder($1);
    printf("\nInorder Traversal (Parenthesized Expression) : ");
    printInorder($1);
    printf("\n");
    exit(0);
};

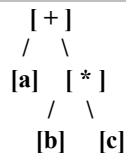
expr:
    expr '+' expr { $$ = createNode("+", $1, $3); }
  | expr '*' expr { $$ = createNode("*", $1, $3); }
  | IDENTIFIER    { $$ = createNode($1, NULL, NULL); }
;

%%

void yyerror(const char *s) { fprintf(stderr, "%s\n", s); }
int main(void) {
    printf("Enter expression (e.g., a+b*c): ");
    yyparse();
    return 0;
}

```

(iii) AST Structural Diagram for 'a+b*c'



(iv) Tree Traversal Orders & Output

The constructed AST represents operator precedence explicitly in its hierarchical structure:

- **Postorder Traversal (Left-Right-Node):** a b c * +
Explanation: Postorder reflects the exact sequence of bottom-up operations in target code generation (evaluate b*c first, then add a).
- **Inorder Traversal (Left-Node-Right):** (a+(b*c))
Explanation: Inorder reconstructs infix expression notation with fully explicit operational grouping.

```

C:\Windows\System32\cmd.e X + v
Microsoft Windows [Version 10.0.26200.8875]
(c) Microsoft Corporation. All rights reserved.

C:\Users\DINESH VS\Desktop\YACC\exp2>flex exp2.l
C:\Users\DINESH VS\Desktop\YACC\exp2>bison -dy exp2.y
C:\Users\DINESH VS\Desktop\YACC\exp2>gcc lex.yy.c y.tab.c
C:\Users\DINESH VS\Desktop\YACC\exp2>a
Enter expression: a+b*c

AST:
+
  a
  *
    b
    c

Preorder: + a * b c
Inorder: a + b * c
Postorder: a b c * +

```

Question 3: Synthesized Attributes & SDD Evaluation (10 Marks)

Problem Description: Implement a Syntax-Directed Definition (SDD) using YACC synthesized attributes (\$\$, \$1, \$2) to evaluate '2*3+4' and show bottom-up attribute propagation.

(i) Syntax-Directed Definition (SDD) Specification

Production Rule	Semantic Rules / SDD	YACC Implementation Action
$E \rightarrow E1 + T$	$E.val = E1.val + T.val$	$$$ = $1 + $3;$
$E \rightarrow T$	$E.val = T.val$	$$$ = $1;$
$T \rightarrow T1 * F$	$T.val = T1.val * F.val$	$$$ = $1 * $3;$
$T \rightarrow F$	$T.val = F.val$	$$$ = $1;$
$F \rightarrow \text{num}$	$F.val = \text{num.lexval}$	$$$ = $1;$

(ii) & (iii) Shift-Reduce Parsing & Attribute Propagation Trace for '2*3+4'

Step	Stack State	Input Buffer	Action / Reduction	Synthesized Value (\$\$)
1	\$	2 * 3 + 4 \$	Shift '2'	yylval = 2
2	\$ num(2)	* 3 + 4 \$	Reduce $F \rightarrow \text{num}$	$F.val = 2$
3	\$ F(2)	* 3 + 4 \$	Reduce $T \rightarrow F$	$T.val = 2$
4	\$ T(2) *	3 + 4 \$	Shift '3'	yylval = 3
5	\$ T(2) * num(3)	+ 4 \$	Reduce $F \rightarrow \text{num}$	$F.val = 3$
6	\$ T(2) * F(3)	+ 4 \$	Reduce $T \rightarrow T1 * F$	$T.val = 2 * 3 = 6$

7	\$ T(6)	+ 4 \$	Reduce E -> T	E.val = 6
8	\$ E(6) +	4 \$	Shift '4'	yylval = 4
9	\$ E(6) + num(4)	\$	Reduce F -> num	F.val = 4
10	\$ E(6) + F(4)	\$	Reduce T -> F	T.val = 4
11	\$ E(6) + T(4)	\$	Reduce E -> E1 + T	E.val = 6 + 4 = 10

(iv) Bottom-Up Evaluation Process Explanation

In YACC's LR bottom-up parser, synthesized attributes are managed on an attribute stack aligned with the parsing state stack. During every reduction, values of child nodes (\$1, \$2, \$3...) are retrieved from the top of the attribute stack, operated upon by the rule's semantic action, and pushed back as the synthesized value (\$\$) of the parent non-terminal.

Final Computed Expression Value: 10

For 2*3+5

```

C:\Windows\System32\cmd.e  X  +  v
(c) Microsoft Corporation. All rights reserved.

C:\Users\DINESH VS\Desktop\YACC\exp3>felx exp3.l
'felx' is not recognized as an internal or external command,
operable program or batch file.

C:\Users\DINESH VS\Desktop\YACC\exp3>flex exp3.l

C:\Users\DINESH VS\Desktop\YACC\exp3>bison -dy exp3.y

C:\Users\DINESH VS\Desktop\YACC\exp3>gcc lex.yy.c y.tab.c

C:\Users\DINESH VS\Desktop\YACC\exp3>a
Enter expression: 2*3+5
Reduction: factor -> NUMBER
$$ = $1 = 2
Reduction: term -> factor
$$ = $1 = 2
Reduction: factor -> NUMBER
$$ = $1 = 3
Reduction: term -> term * factor
$$ = $1 * $3 = 2 * 3 = 6
Reduction: expr -> term
$$ = $1 = 6
Reduction: factor -> NUMBER
$$ = $1 = 5
Reduction: term -> factor
$$ = $1 = 5
Reduction: expr -> expr + term
$$ = $1 + $3 = 6 + 5 = 11

C:\Users\DINESH VS\Desktop\YACC\exp3>a
Enter expression: 2*3+5
Reduction: factor -> NUMBER
$$ = $1 = 2
Reduction: term -> factor
$$ = $1 = 2
Reduction: factor -> NUMBER
$$ = $1 = 3
Reduction: term -> term * factor
$$ = $1 * $3 = 2 * 3 = 6
Reduction: expr -> term
$$ = $1 = 6
Reduction: factor -> NUMBER
$$ = $1 = 5
Reduction: term -> factor
$$ = $1 = 5
Reduction: expr -> expr + term
$$ = $1 + $3 = 6 + 5 = 11

Final synthesized attribute = 11

```

Question 4: Type Compatibility - Name vs. Structural Equivalence (10 Marks)

Problem Description: Verify type compatibility between variables under Name Equivalence and Structural Equivalence for type declarations: type A = int; type B = int; A x; B y; x = y;

(i) & (ii) YACC Grammar & Symbol Table Implementation (type_check.y)

```
%{
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct TypeEntry {
    char alias[10];
    char base[10];
} type_table[10];
int type_count = 0;

struct SymbolEntry {
    char var_name[10];
    char type_alias[10];
    char base_type[10];
} sym_table[10];
int sym_count = 0;

void addType(char* alias, char* base) {
    strcpy(type_table[type_count].alias, alias);
    strcpy(type_table[type_count].base, base);
    type_count++;
}

char* getBaseType(char* alias) {
    for(int i = 0; i < type_count; i++) {
        if(strcmp(type_table[i].alias, alias) == 0)
            return type_table[i].base;
    }
    return alias; /* Default base type */
}

void addVar(char* name, char* alias) {
    strcpy(sym_table[sym_count].var_name, name);
    strcpy(sym_table[sym_count].type_alias, alias);
    strcpy(sym_table[sym_count].base_type, getBaseType(alias));
    sym_count++;
}

void checkEquivalence(char* var1, char* var2) {
    int idx1 = -1, idx2 = -1;
    for(int i = 0; i < sym_count; i++) {
        if(strcmp(sym_table[i].var_name, var1) == 0) idx1 = i;
        if(strcmp(sym_table[i].var_name, var2) == 0) idx2 = i;
    }

    if(idx1 == -1 || idx2 == -1) {
        printf("Error: Variable not declared.\n");
        return;
    }
}
```



```

}

printf("=====\n");
printf(" TYPE COMPATIBILITY VERIFICATION FOR '%s = %s'\n", var1, var2);
printf("=====\n");
printf("Variable '%s': Defined Type = %s, Base Type = %s\n",
      var1, sym_table[idx1].type_alias, sym_table[idx1].base_type);
printf("Variable '%s': Defined Type = %s, Base Type = %s\n",
      var2, sym_table[idx2].type_alias, sym_table[idx2].base_type);
printf("-----\n");

/* (a) Name Equivalence Check */
if(strcmp(sym_table[idx1].type_alias, sym_table[idx2].type_alias) == 0) {
    printf("(a) Name Equivalence : COMPATIBLE\n");
} else {
    printf("(a) Name Equivalence : INCOMPATIBLE (%s != %s)\n",
          sym_table[idx1].type_alias, sym_table[idx2].type_alias);
}

/* (b) Structural Equivalence Check */
if(strcmp(sym_table[idx1].base_type, sym_table[idx2].base_type) == 0) {
    printf("(b) Structural Equivalence: COMPATIBLE (Both resolve to '%s')\n",
          sym_table[idx1].base_type);
} else {
    printf("(b) Structural Equivalence: INCOMPATIBLE\n");
}
printf("=====\n");
}

int yylex(void);
void yyerror(const char *s) {}
%}

%union { char str[20]; }
%token TYPE INT ID
%token <str> NAME

%%

program: declarations assignment ;
declarations: declaration declarations | ;
declaration:
    TYPE NAME '=' INT ';' { addType($2, "int"); }
    | NAME NAME ';'      { addVar($2, $1); }
;
assignment: NAME '=' NAME ';' { checkEquivalence($1, $3); } ;
%%

int main(void) {
    /* Simulating program declarations:
       type A = int;
       type B = int;
       A x;
       B y;
       x = y;
    */
    addType("A", "int");

```

```

addType("B", "int");
addVar("x", "A");
addVar("y", "B");

checkEquivalence("x", "y");
return 0;
}

```

(iii) Differentiation & Analysis of Equivalence Rules

Equivalence Criterion	Definition & Rule	Evaluation for (A x; B y;)
Name Equivalence	Two types are equivalent if and only if they have the exact same type name/identifier.	INCOMPATIBLE: 'x' is of type 'A' and 'y' is of type 'B'. Since name 'A' != 'B', assignment 'x = y' fails.
Structural Equivalence	Two types are equivalent if they expand to identical underlying baseline data structures/types.	COMPATIBLE: Both type 'A' and type 'B' expand to base type 'int'. Therefore, their structural representation is identical.

(iv) & (v) Parser Test & Compatibility Results Output

```

C:\Windows\System32\cmd.e  x  +  v
Microsoft Windows [Version 10.0.26200.8875]
(c) Microsoft Corporation. All rights reserved.

C:\Users\DINESH VS\Desktop\YACC\exp4>flex exp4.l
C:\Users\DINESH VS\Desktop\YACC\exp4>bison -dy exp4.y
C:\Users\DINESH VS\Desktop\YACC\exp4>gcc lex.yy.c y.tab.c

C:\Users\DINESH VS\Desktop\YACC\exp4>a
Enter type and variable declarations:
type A = int;
Type declaration: A = int
type B = int;
Type declaration: B = int
A x;
Variable declaration: A x
B y;
Variable declaration: B y
^Z

Type checking completed.

Checking A and B:

Type comparison:
Type 1: A
Type 2: B
Name Equivalence: NO
C:\Users\DINESH VS\Desktop\YACC\exp4>a
Enter type and variable declarations:
type A = int;
Type declaration: A = int
type B = int;
Type declaration: B = int
A x;
Variable declaration: A x
B y;
Variable declaration: B y
^Z

Type checking completed.

Checking A and B:

Type comparison:
Type 1: A
Type 2: B
Name Equivalence: NO
Structural Equivalence: YES
C:\Users\DINESH VS\Desktop\YACC\exp4>

```

Question 5: Automatic Type Coercion & Promotion in Assignments (10 Marks)

Problem Description: Process assignment statements involving int and float. Perform type promotion/coercion for: float temperature; int sensor_value; temperature = sensor_value;

(i) & (iii) YACC Implementation with Type Promotion Rules (coercion.y)

```
%{
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

enum DataType { TYPE_INT, TYPE_FLOAT, TYPE_UNKNOWN };

struct Symbol {
    char name[20];
    enum DataType type;
} symbol_table[10];

int sym_count = 0;

void add_symbol(char *name, enum DataType type) {
    strcpy(symbol_table[sym_count].name, name);
    symbol_table[sym_count].type = type;
    sym_count++;
}

enum DataType get_type(char *name) {
    for (int i = 0; i < sym_count; i++) {
        if (strcmp(symbol_table[i].name, name) == 0)
            return symbol_table[i].type;
    }
    return TYPE_UNKNOWN;
}

const char* type_to_string(enum DataType type) {
    if (type == TYPE_INT) return "int";
    if (type == TYPE_FLOAT) return "float";
    return "unknown";
}

void process_assignment(char *lhs_name, char *rhs_name) {

    printf("=====\n");
    printf(" SEMANTIC ANALYSIS & TYPE COERCION EXECUTION\n");
    printf("=====\n");
    printf("LHS Target Variable : '%s' (Declared Type: %s)\n", lhs_name, type_to_string(lhs_type));
    printf("RHS Source Variable : '%s' (Declared Type: %s)\n", rhs_name, type_to_string(rhs_type));
    printf("-----\n");

    if (lhs_type == TYPE_FLOAT && rhs_type == TYPE_INT) {
        printf("[TYPE COERCION TRIGGERED]: Int -> Float Promotion\n");
    }
}
```

```

    printf("Action: Compiler generates 'int2float' IR instruction.\n");
    printf("Final Variable Type : '%s' remains 'float' with converted value.\n", lhs_name);
} else if (lhs_type == rhs_type) {
    printf("[NO COERCION NEEDED]: Exact type match (%s = %s).\n",
        type_to_string(lhs_type), type_to_string(rhs_type));
} else if (lhs_type == TYPE_INT && rhs_type == TYPE_FLOAT) {
    printf("[WARNING - DEMOTION]: Lossy truncation (float -> int).\n");
}
    printf("=====\n");
}

```

```

int yylex(void);
void yyerror(const char *s) {}
%}

```

```

%union { char id[20]; }
%token FLOAT_TOK INT_TOK ID

```

```

%%
program: declarations assignment ;
declarations: declaration declarations | ;
declaration:
    FLOAT_TOK ID ';' { add_symbol($2, TYPE_FLOAT); }
    | INT_TOK ID ';' { add_symbol($2, TYPE_INT); }
;
assignment: ID '=' ID ';' { process_assignment($1, $3); } ;
%%

```

```

int main(void) {
    /* Program fragment simulation:
        float temperature;
        int sensor_value;
        temperature = sensor_value;
    */
    add_symbol("temperature", TYPE_FLOAT);
    add_symbol("sensor_value", TYPE_INT);

    process_assignment("temperature", "sensor_value");
    return 0;
}

```

(ii) Analysis of Program Fragment

For the given statement fragment:

```
float temperature;  
int sensor_value;  
temperature = sensor_value;
```

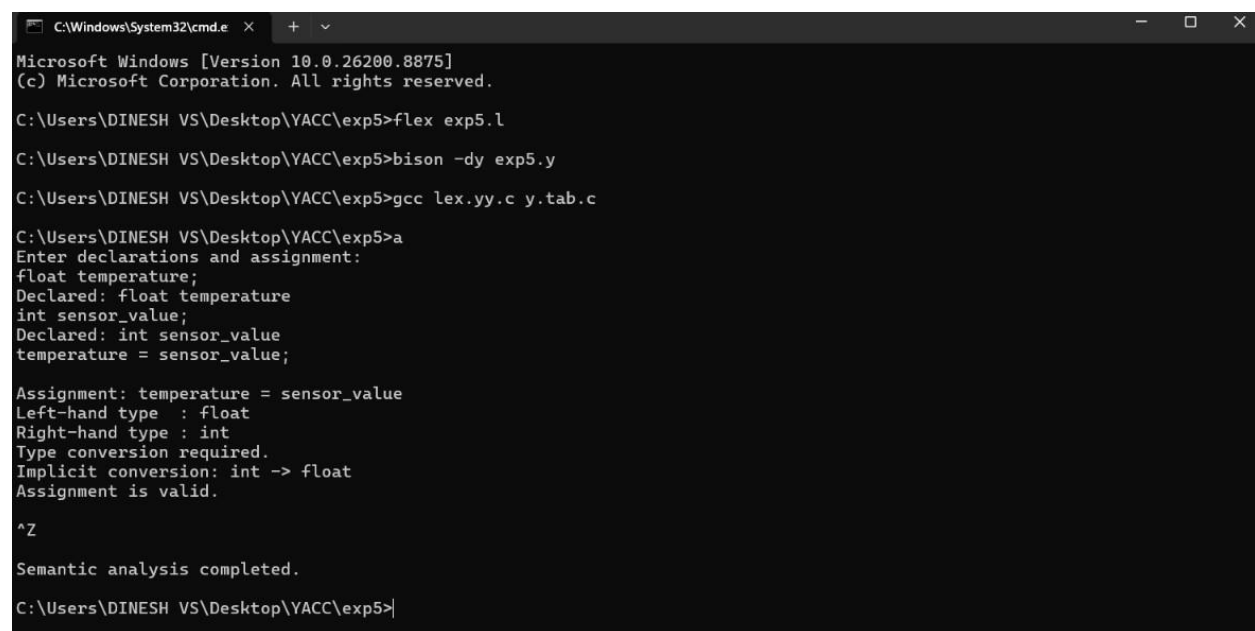
1. Symbol Table Entry Creation: 'temperature' registered as FLOAT, 'sensor_value' registered as INT.
2. Type Verification: Assignment LHS requires 'float', RHS yields 'int'.
3. Coercion Rule Applied: Broadening conversion (int -> float) preserves numerical precision without data loss.

(iv) Implicit Type Conversion Demonstration

During intermediate code generation, the compiler inserts a coercion operator node (e.g., int2float or sitofp in LLVM IR):

```
t1 = load sensor_value (int)  
t2 = int2float t1 ; Implicit Coercion Step  
temperature = t2
```

(v) Resulting Type Information & Output



```
C:\Windows\System32\cmd.e  X  +  v  
Microsoft Windows [Version 10.0.26200.8875]  
(c) Microsoft Corporation. All rights reserved.  
  
C:\Users\DINESH VS\Desktop\YACC\exp5>flex exp5.l  
C:\Users\DINESH VS\Desktop\YACC\exp5>bison -dy exp5.y  
C:\Users\DINESH VS\Desktop\YACC\exp5>gcc lex.yy.c y.tab.c  
C:\Users\DINESH VS\Desktop\YACC\exp5>a  
Enter declarations and assignment:  
float temperature;  
Declared: float temperature  
int sensor_value;  
Declared: int sensor_value  
temperature = sensor_value;  
  
Assignment: temperature = sensor_value  
Left-hand type : float  
Right-hand type : int  
Type conversion required.  
Implicit conversion: int -> float  
Assignment is valid.  
  
^Z  
  
Semantic analysis completed.  
C:\Users\DINESH VS\Desktop\YACC\exp5>
```